Machine Learning Project: Using Multi-Class Classification and One-Vs-All Approaches to Processed Interaction Data

DS 5640 – Machine Learning | Spring 2026

Vanderbilt University

Suyash Deshmukh Daniel Gormezano Yi He

Israel Lwamba Payton Robinette Jiho Yoo

April 19, 2026

Abstract

This project addresses multi-class classification of user interface interaction actions using pre-processed, feature-based tabular data derived from frame sequences. The dataset contains 11 original action classes with considerable class imbalance; six rare classes are consolidated into an *other* category, yielding 6 effective classes: click, drag, other, select, zoom-in, and zoom-out. We compare two modeling strategies: a One-vs-All (OvA) approach using Elastic Net logistic regression, and direct multi-class classification using Random Forest and XGBoost. Class imbalance is addressed through SMOTE oversampling, threshold tuning, and class-weight rebalancing. XGBoost achieves the best overall performance with a weighted F1 of 0.71, while the OvA approach attains a macro F1 of 0.50. Tree-based ensemble methods outperform the linear approach across all metrics, with class imbalance remaining a persistent challenge for minority classes.

1 Introduction

1.1 Data Description

The dataset used in this project consists of processed, feature-based tabular data that were split into training (`X_train` and `y_train`) and testing sets (`X_test` and `y_test`). Each row represents a single user action derived from a sequence of three consecutive frames, and each of the 147 columns represents frame-level properties and temporal changes between frames. The dataset contains 11 action classes, which include drag, click, zoomin and zoomout, select, type, hover, scrolldown, appear, disappear, and scrollup.

As shown in Figure 1, the class distribution is very imbalanced, with drag and click dominating the dataset (44.9% and 32.6%, respectively), whereas many classes (scrolldown, appear, disappear, and scrollup) occur in less than 1% of samples. To address this imbalance, all classes with frequencies below *select* were grouped into an *other* category. This resulted in a more stable distribution where *other* represents 4.1% of the data. This consolidation helps improve model learning and evaluation by reducing the extreme sparsity in rare classes.

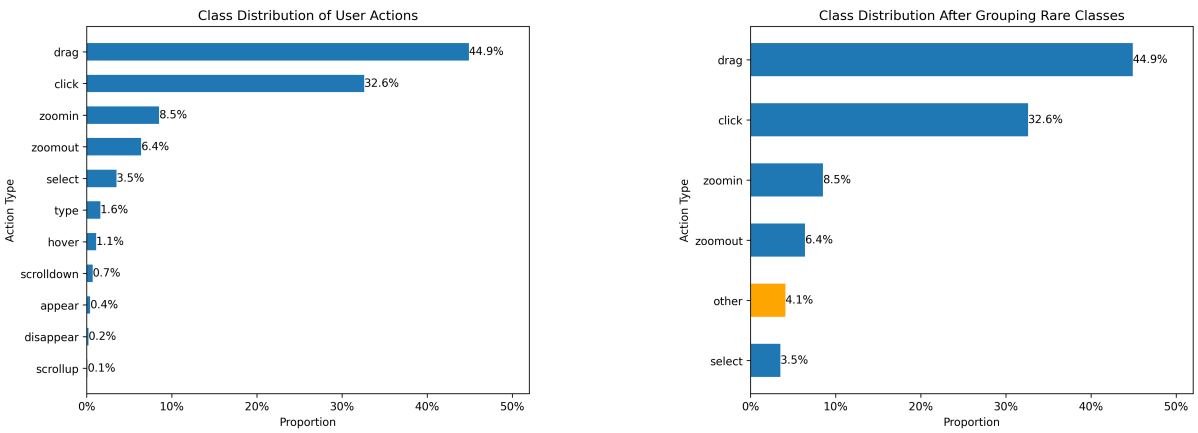

Figure 1: Class distribution before (left) and after (right) consolidating rare classes into the *other* category.

2 Problem Formulation

To address the multi-class problem, our first approach was implementing a One-vs-All (OvA) classification framework. In this framework, a separate binary classifier was trained for each

action class, where the target class was treated as the positive outcome (1) and all other classes were treated as the negative outcome (0). After grouping the rare classes into another category, this resulted in six OvA models: Click, Drag, Zoom-in, Zoom-out, Select, and Other. Each model was trained using Elastic Net logistic regression, combining L1 and L2 regularization to handle the highly dimensional feature space and reduce overfitting. To address class imbalance, SMOTE was applied to each binary training set. Classification thresholds were also tuned on a validation split to maximize the F1 score rather than relying on the default 0.5 threshold, improving the tradeoff between precision and recall. The six binary models were combined into a final multi-class model that selects the class with the highest predicted probability for each sample. This approach allows the model to apply class-specific decision boundaries and produce interpretable multi-class predictions.

Our second approach was a direct multi-class classification strategy using Random Forest and XGBoost. Random Forest is an ensemble method that constructs multiple decision trees using bootstrap sampling and random feature selection, and aggregates their predictions to improve generalization and reduce overfitting. In contrast, XGBoost is a gradient boosted ensemble method that builds decision trees sequentially, where each tree corrects the errors of the previous one. Both approaches are well-suited for imbalanced data with strong performance on structured feature spaces and can capture complex non-linear relationships. Rather than training separate binary classifiers, these models were trained to predict all six action classes simultaneously. To address class imbalance, SMOTE oversampling was applied to the training set prior to model fitting. Hyperparameters were optimized using GridSearchCV with 5-fold cross-validation, and the models were evaluated using accuracy, precision, recall, and macro F1-score.

3 Handling Class Imbalance

This dataset exhibits considerable class imbalance, with the actions drag and click dominating over 75% of the distribution collectively, while other actions appeared infrequently. To address this, a combination of data-level and modeling-level techniques were applied.

3.1 One-vs-All Approach

For the OvA model, rare classes with very low representation (type, hover, scrolldown, appear, disappear, and scrollup) were first grouped into an *other* category that reduced sparsity and ensured each class had a sufficient number of samples for learning. For each binary OvA model, SMOTE oversampling was applied to the training data to produce synthetic examples of the minority class, which improved class balance without discarding useful signals and allowed each classifier to better learn the decision boundaries for underrepresented actions.

3.2 Multi-Class Classification Approach

For both the Random Forest and the XGBoost multi-class models, the same six-class grouping was applied, consolidating the six rare classes into the *other* category. SMOTE oversampling was applied within a pipeline prior to training to synthetically balance the class distribution, ensuring that the model did not become biased toward the dominant drag and click classes during learning. For the Random Forest model, we also explored `balanced` class weights to increase the weight of minority classes; however, introducing class weights reduced model performance, so it was removed from the final configuration.

4 Models and Approaches

4.1 One-vs-All (OvA) Model

For the OvA model, we trained six binary logistic regression classifiers. To adjust for class imbalance, the original 11 classes were reduced to six by grouping the rare classes into a single *other* category. We then paired our six classifiers with SMOTE oversampling to address observed class imbalance. All input features were standardized using `StandardScaler` (z-score normalization) so that each feature has a mean of 0 and a variance of 1, ensuring the model treats all features on the same scale.

We tuned each of the six models using an Elastic Net regression by searching over C (the inverse regularization strength, controlling model fit and simplicity) and `l1_ratio` (the balance between L1 sparsity and L2 stability) to optimize the precision-recall tradeoff for each class. Threshold tuning was also applied on the validation data by selecting class-specific probability cutoffs instead of relying on the default 0.5 threshold, to better balance precision and recall. As a final step, the six binary classifiers were combined into a single multi-class prediction by assigning each sample to the class with the highest probability. Macro F1, precision, and recall were recorded to evaluate model performance.

4.2 Random Forest (Multi-Class) Model

For the multi-class Random Forest model, we trained a single classifier to predict all six action classes simultaneously. Our Random Forest constructs an ensemble of decision trees using bootstrap sampling and random feature selection at each split, enabling it to capture non-linear relationships and interactions in the feature space while reducing overfitting through aggregation. To further address class imbalance, SMOTE oversampling was applied within a pipeline so that synthetic samples were generated only on the training folds during cross-validation. We also included `class_weight` as a tunable parameter to allow the model to adjust for class imbalance internally.

Hyperparameters, including the number of trees, maximum tree depth, minimum samples per split and leaf, and feature subsampling, were optimized using GridSearchCV with 5-fold cross-validation and `f1_macro` as the scoring metric to ensure equal emphasis across all classes. The best-performing configuration used 300 trees with a maximum depth of 10, `max_features=''sqrt''`, `min_samples_split=10`, and no class weighting, achieving a cross-validation `f1_macro` score of 0.4707.

4.3 XGBoost (Multi-Class) Model

For the multi-class model, we trained a single XGBoost classifier on all six action classes simultaneously. XGBoost uses gradient boosting over decision trees, iteratively fitting new trees to the residuals of prior predictions, which makes it robust to non-linear relationships and feature interactions present in the bounding-box feature space. The model used a `multi:softmax` objective for multi-class classification with log-loss (`mlogloss`) as the evaluation metric. To handle class imbalance, SMOTE was applied within a pipeline prior to training.

Hyperparameters were tuned using GridSearchCV with 5-fold cross-validation and `f1_macro` as the scoring metric. The best configuration found was `n_estimators=100`, `max_depth=3`, `learning_rate=0.1`, `subsample=0.8`, and `colsample_bytree=1.0`, achieving a best cross-validation `f1_macro` score of 0.4841.

5 Evaluation Results

5.1 One-vs-All Model

Table 1 reveals the combined multi-class performance metrics of the OvA model. Table 2 shows the per-class performance metrics for each individual action class. In addition to these evaluation metrics, the hyperparameters for each binary classifier were optimized using GridSearchCV. The regularization strength parameter (C) and elastic net mixing parameter (`l1_ratio`) were tuned separately for each class to balance model flexibility and sparsity. The values are as follows: $C = 1$ and `l1_ratio` $= 0.7$ for click; $C = 10$ and `l1_ratio` $= 0.7$ for drag; $C = 10$ and `l1_ratio` $= 0.3$ for zoom-in; $C = 10$ and `l1_ratio` $= 0.5$ for zoom-out; $C = 10$ and `l1_ratio` $= 0.3$ for other; and $C = 10$ and `l1_ratio` $= 0.3$ for select.

Table 1: OvA Model: Combined Multi-Class Performance

Accuracy	**Precision (macro)**	**Recall (macro)**	**F1-Score (macro)**
0.5596	0.4783	0.5975	0.4969

Table 2: OvA Model: Per-Class Performance Metrics

Class	**Precision**	**Recall**	**F1-Score**
Click	0.55	0.49	0.52
Drag	0.75	0.54	0.63
Other	0.14	0.50	0.22
Select	0.12	0.48	0.19
Zoom-in	0.61	0.84	0.71
Zoom-out	0.70	0.73	0.72

5.2 Random Forest (Multi-Class) Model

Table 3 presents the overall multi-class performance metrics for the Random Forest model. Table 4 shows the per-class breakdown of precision, recall, and F1-score for each of the six action classes.

Table 3: Random Forest: Overall Multi-Class Performance

Accuracy	**Precision (weighted)**	**Recall (weighted)**	**F1-Score (weighted)**
0.6856	0.7315	0.6856	0.6936

Table 4: Random Forest: Per-Class Classification Report

Class	**Precision**	**Recall**	**F1-Score**
Click	0.59	0.72	0.65
Drag	0.84	0.60	0.70
Other	0.47	0.80	0.59
Select	0.25	0.61	0.36
Zoom-in	0.78	0.91	0.84
Zoom-out	0.86	0.84	0.85

5.3 XGBoost (Multi-Class) Model

Table 5 presents the overall multi-class performance metrics for the XGBoost model. Table 6 shows the per-class breakdown of precision, recall, and F1-score for each of the six action classes.

Table 5: XGBoost: Overall Multi-Class Performance

Accuracy	**Precision (weighted)**	**Recall (weighted)**	**F1 (weighted)**
0.6934	0.7422	0.6934	0.7062

Table 6: XGBoost: Per-Class Classification Report

Class	**Precision**	**Recall**	**F1-Score**
Click	0.65	0.71	0.68
Drag	0.85	0.64	0.73
Other	0.25	0.65	0.37
Select	0.25	0.57	0.34
Zoom-in	0.84	0.90	0.87
Zoom-out	0.70	0.84	0.76

6 Results Analysis

6.1 One-vs-All (OvA) Model

6.1.1 Combined Multi-Class Performance

We used macro F1 because it accounts for class imbalance by averaging the F1 scores across classes, which guarantees that performance on minority classes is not overshadowed by majority classes. The OvA model demonstrated moderate overall performance (macro F1 = 0.49), with substantial heterogeneity across classes. Performance was strongest for interaction types that appear to have more distinct behavioral signatures, such as zoom-in, zoom-out, and drag, all of which achieved F1 scores above 0.60. In contrast, the *other* and *select* categories exhibited notably poor performance, with F1 scores below 0.25.

Moreover, macro recall (0.59) exceeds macro precision (0.47). This indicates that the model is less conservative overall, meaning it identifies more true instances, but at the cost of making more false positive errors across classes.

6.1.2 Per-Class Performance

The zoom-out and zoom-in classifiers are the strongest, both achieving F1 scores above 0.70. Zoom-out shows well-balanced precision (0.70) and recall (0.73), meaning that it correctly identifies most actual zoom-out actions while keeping false positives manageable. Zoom-in achieves the highest recall of any class (0.84), catching the vast majority of true zoom-in actions. However, its precision is somewhat lower (0.61). These two classes likely have distinctive features that the logistic regression model can separate cleanly from the rest.

The drag classifier was a moderate performer, with an F1 of 0.63, precision of 0.75, and recall of 0.54. Of note, its precision was the highest of any class, indicating that when the model predicts drag, it is correct nearly three-quarters of the time. However, its recall of only 0.54 means that it misses nearly half of all true drag actions.

Click achieves an F1 of 0.52, with moderate precision (0.55) and recall (0.49). The model misses about half of actual click actions and produces a notable number of false positives. Click actions likely share similar feature patterns with other actions, making them harder to isolate.

The *other* and *select* classes perform the worst, with F1 scores of 0.22 and 0.19 respectively. Both share a similar pattern: moderate recall (0.50 and 0.48) and very low precision (0.14 and 0.12). This means that while these two models detect roughly half of true instances, they do so at the cost of producing considerable false positives.

6.2 Random Forest Model

6.2.1 Combined Multi-Class Performance

We used the weighted F1-score as the primary evaluation metric for the Random Forest model because it accounts for class imbalance by weighting each class according to its sample size. This ensures that overall performance reflects both majority and minority class contributions. The Random Forest model demonstrated slightly improved overall performance, achieving an accuracy of 0.6856, a weighted precision of 0.7315, a weighted recall of 0.6856, and a weighted F1-score of 0.6936.

Performance varied across classes, with stronger results observed for interaction types that have more distinct behavioral patterns. In particular, zoom-in and zoom-out achieved the highest performance, with F1-scores of 0.84 and 0.85 respectively. These results suggest that zoom-related actions have clearer distinguishing features that the Random Forest model can effectively capture. In contrast, minority classes such as select and other showed lower performance, with F1-scores of 0.36 and 0.59 respectively, indicating that these categories remain challenging to classify accurately due to class imbalance and overlapping behavioral characteristics. It should be noted that the Random Forest achieved the highest F1-score for the *other* class of all three model types.

Additionally, the similarity between weighted recall (0.6856) and accuracy (0.6856) indicates that the model maintains balanced detection of true instances across classes. The slightly higher weighted precision (0.7315) suggests that the model produces relatively fewer false positive predictions overall compared to missed detections.

6.2.2 Per-Class Performance

Among all classes, zoom-out and zoom-in demonstrated the strongest classification performance. The zoom-out classifier achieved an F1-score of 0.85, supported by strong precision (0.86) and recall (0.84), indicating that the model correctly identified most zoom-out actions while maintaining low false positive rates. Similarly, zoom-in achieved an F1-score of 0.84, with precision of 0.78 and recall of 0.91. The particularly high recall suggests that the majority of true zoom-in actions were successfully detected, although some false positives were introduced due to slightly lower precision.

The drag class showed moderate performance, achieving an F1-score of 0.70, with precision of 0.84 and recall of 0.60. Notably, drag exhibited relatively high precision, indicating that predictions labeled as drag were frequently correct. However, the lower recall suggests that some true drag actions were missed by the model.

The click class achieved moderate performance, with an F1-score of 0.65, supported by precision of 0.59 and recall of 0.72. The higher recall compared to precision indicates that the model successfully captured many true click actions but also produced some false positive predictions.

The *other* class demonstrated mixed performance, with an F1-score of 0.59, precision of 0.47,

and recall of 0.80. The relatively high recall suggests that the model detected most true *other* instances, but the lower precision indicates substantial confusion with other classes.

The *select* class exhibited the weakest performance among all categories, with an F1-score of 0.36, precision of 0.25, and recall of 0.61. The very low precision indicates that many predicted *select* instances were incorrect, suggesting that this class shares overlapping features with other interaction types and remains difficult for the model to distinguish reliably.

6.3 XGBoost (Multi-Class) Model

6.3.1 Combined Multi-Class Performance

The XGBoost model achieved a weighted F1 of 0.71, the highest among all models evaluated in this project. We used weighted F1 as the primary evaluation metric for XGBoost (consistent with the Random Forest), as it accounts for class imbalance by weighting each class according to its sample size. Weighted precision (0.74) exceeds weighted recall (0.69), indicating the model is generally conservative: when it commits to a prediction, it tends to be correct, but it misses a portion of true instances, particularly minority classes. The overall accuracy of 0.6934 reflects the model's solid handling of the dominant classes while being constrained by the limited representation of *other* and *select*.

6.3.2 Per-Class Performance

The zoom-in class achieved the highest F1 of 0.87, with strong precision (0.84) and excellent recall (0.90), meaning the model correctly identifies nearly all true zoom-in interactions with few false positives. Zoom-out followed with an F1 of 0.76, showing well-balanced precision (0.70) and recall (0.84). These two classes likely have the most distinctive spatial signatures in the bounding-box features, making them the easiest to separate from other actions.

Drag achieved an F1 of 0.73 with the highest precision of any class (0.85), meaning that when the model predicts drag it is almost always correct. However, its recall of 0.64 indicates it misses roughly one-third of true drag instances. Click achieved an F1 of 0.68, with a more balanced precision (0.65) and recall (0.71).

The *other* and *select* classes remain the weakest, with F1 scores of 0.37 and 0.34 respectively. Both share a pattern of low precision (0.25) paired with moderate recall, meaning the model over-predicts these classes relative to how often they truly occur. This is an expected outcome given the very small number of test samples for these classes (20 and 23 respectively) and the fact that *other* is a heterogeneous catch-all category.

7 Conclusion and Future Work

The results of this project demonstrate that tree-based ensemble methods considerably outperform linear approaches across all evaluation metrics, with XGBoost achieving the highest weighted F1 of nearly 0.71, compared to the OvA approach with a macro F1 of 0.49. This performance gap is expected, as gradient boosted and ensemble methods are better at capturing the non-linear relationships and feature interactions present in the dataset analyzed, whereas logistic regression is constrained by its linear separability assumption. Across all models, class imbalance was a persistent challenge: the zoom-in and zoom-out classes consistently achieved the best performance due to their separable feature profiles, while the *other* and *select* categories remained the weakest across every method. This reflects the difficulty of classifying heterogeneous and low-frequency actions, even after applying class grouping and imbalance-handling methods such as SMOTE.

There are several implications for this project. Although SMOTE and class consolidation mitigated some of the severe effects of class imbalance, neither were able to fully ameliorate the precision deficits observed for minority classes. This suggests that techniques such as cost-sensitive learning may be needed to meaningfully improve performance on rare classes. Additionally, threshold tuning in the OvA approach improved the precision-recall tradeoff for individual classifiers, but the gains did not translate into improved aggregate performance compared to the tree-based models. This elucidates how class-specific decision boundaries did not outweigh the architectural advantages of tree-based models.

Future directions of this project would involve exploring temporal models that treat frame sequences as time series data instead of static differences. Additionally, cost-sensitive learning and other class imbalance approaches that penalize misclassification of minority classes more heavily during training could be used.

Figure 2 summarizes the per-class F1 scores across all three models, illustrating where each approach performs strongly and where it falls short.

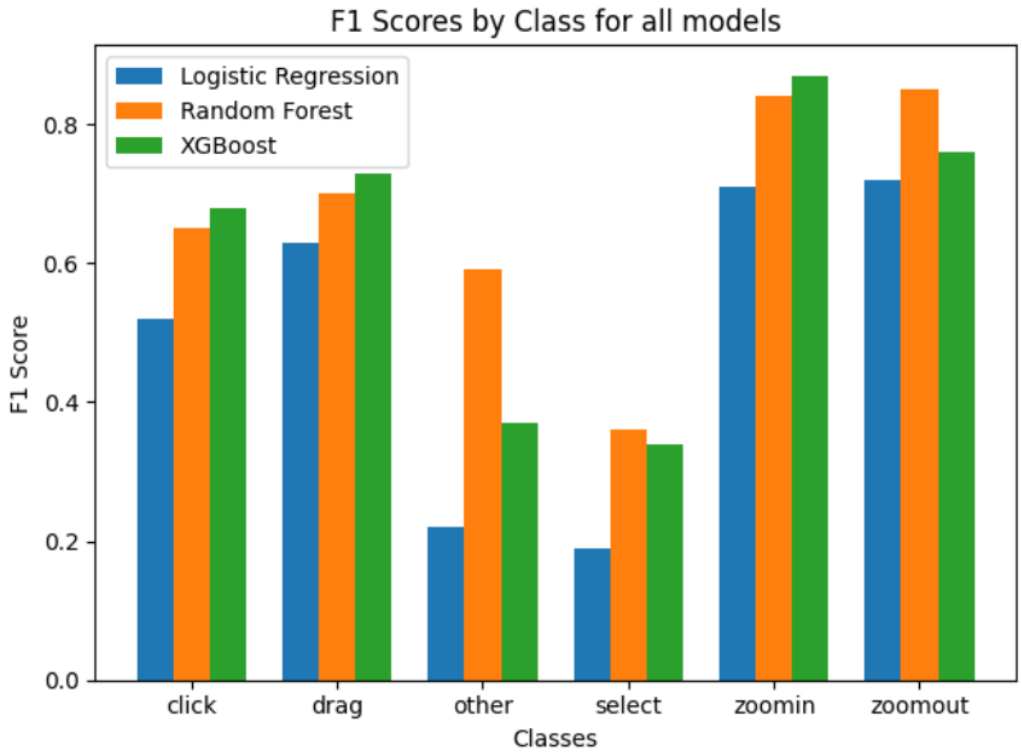

Figure 2: F1 scores by class for all three models. OvA uses macro F1; Random Forest and XGBoost use weighted F1.

Appendices

Appendix A: GitHub Repository

This appendix contains a link to the GitHub repository used to organize all project files and data. The repository contains all the code used to build, train, and evaluate the multi-class classification models and one-vs-all models for this project.

https://github.com/Daniel-Gormezano/ds5640-machine-learning-project/tree/main